

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

СОГЛАСОВАНО
Научно-учебная лаборатория
методов анализа больших
данных
Стажер-исследователь

_____ / М.В. Минец /
«__» _____ 2026 г.

УТВЕРЖДАЮ
Академический руководитель
образовательной программы
«Программная инженерия»,
старший преподаватель
департамента программной
инженерии

_____ / Н.А. Павлочев /
«__» _____ 2026 г.

**СИСТЕМА РЕГИСТРАЦИИ И УПРАВЛЕНИЯ ЖИЗНЕННЫМ ЦИКЛОМ
МЕРОПРИЯТИЙ**

Техническое задание

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.09.01-01 ТЗ 01–1 ЛУ

Исполнители:

студент группы БПИ238
_____ / М.М. Апаркин /
«__» _____ 2026 г.
студент группы БПИ238
_____ / В. Цуркан /
«__» _____ 2026 г.
студент группы БПИ249
_____ / Г.О. Лещук /
«__» _____ 2026 г.

Москва 2026

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подл.	

УТВЕРЖДЕН

RU.17701729.09.01-01 ТЗ 01–1 ЛУ

**СИСТЕМА РЕГИСТРАЦИИ И УПРАВЛЕНИЯ ЖИЗНЕННЫМ ЦИКЛОМ
МЕРОПРИЯТИЙ**

Техническое задание

RU.17701729.09.01-01 ТЗ 01–1

Листов 69

Инов.№ подл.	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	1
1.1. Наименование темы разработки	1
1.2. Наименование программы	1
1.3. Состав комплекса программ	1
1.4. Краткая характеристика области применения	2
2. ОСНОВАНИЯ ДЛЯ РАЗРАБОТКИ	3
2.1. Документы, на основании которых ведётся разработка	3
2.2. Наименование темы разработки	3
3. НАЗНАЧЕНИЕ РАЗРАБОТКИ	4
3.1. Функциональное назначение программы	4
3.2. Эксплуатационное назначение программы	5
3.2.1. Архитектура развёртывания	5
3.2.2. Каналы доступа пользователей	6
3.2.3. Требования к окружению и обслуживанию	6
3.2.4. Сценарии использования	7
4. ТРЕБОВАНИЯ К ПРОГРАММЕ ИЛИ ПРОГРАММНОМУ ИЗДЕЛИЮ	9
4.1. Требования к функциональным характеристикам	9
4.1.1. Подсистема аутентификации и авторизации (auth-manager)	9
4.1.2. Подсистема управления мероприятиями (events-service)	11
4.1.3. Подсистема обработки платежей (payment-service)	15
4.1.4. Подсистема уведомлений (notifier)	17
4.1.5. Подсистема Telegram-бота (tg-bot)	19
4.1.6. Подсистема генерации QR-кодов (qr-service)	20
4.1.7. Веб-интерфейс (frontend)	21
4.1.8. Подсистема мониторинга и наблюдаемости	27
4.1.9. Инфраструктурный и эксплуатационный контур	28
4.2. Требования к надёжности	29
4.3. Условия эксплуатации	30
4.3.1. Климатические условия эксплуатации	30
4.3.2. Требования к видам обслуживания	30
4.3.3. Количество и квалификация пользователей	31
4.4. Требования к составу и параметрам технических средств	31
4.5. Требования к информационной и программной совместимости	32
4.6. Требования к маркировке и упаковке	33
4.7. Требования к транспортированию и хранению	34
4.8. Специальные требования	34
4.8.1. Требования к безопасности	34
4.8.2. Требования к производительности	36

4.8.3. Требования к масштабируемости	36
4.8.4. Предполагаемые параметры нагрузки	37
4.8.5. Требования к CI/CD	37
4.8.6. Требования к журналированию и диагностике	40
4.8.7. Требования к отказоустойчивости	41
4.8.8. Формализованные endpoint-контракты (обязательные требования) ...	41
4.8.9. Матрица ошибок и единый формат отказов	43
4.8.10. State-machine переходы доменных сущностей	44
4.8.11. Сценарии отказов и восстановление	46
4.8.12. Требования к мониторингу, SLI/SLO и алертингу	47
5. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ	49
5.1. Состав программной документации	49
5.2. Специальные требования к программной документации	50
5.2.1. Требования к формату и оформлению	50
5.2.2. Требования к содержанию документации	50
5.2.3. Требования к электронной доставке	51
6. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ	53
6.1. Экономическая эффективность	53
6.1.1. Снижение трудозатрат организаторов	53
7. СТАДИИ И ЭТАПЫ РАЗРАБОТКИ	54
7.1. Сроки разработки и исполнители	54
8. ПОРЯДОК КОНТРОЛЯ И ПРИЁМКИ	55
8.1. Критерии приёмки	55
8.2. Процедура приёмки	56
8.3. Специальные требования к тестированию для events-service	57
8.4. Требования к результатам приёмки	58
Events Service (порт 8000)	63
Auth Manager (порт 8001)	64
Payment Service (порт 8004)	64
Notifier (порт 8002)	64

1. ВВЕДЕНИЕ

1.1. Наименование темы разработки

Наименование темы разработки — «Система регистрации и управления жизненным циклом мероприятий».

1.2. Наименование программы

Наименование программы — «Система регистрации и управления жизненным циклом мероприятий» (далее по тексту — «Платформа», «Система», «CSTATI.events»).

Наименование программы на английском языке — «Event registration and lifetime management system».

Краткое наименование: **CSTATI.events**.

Обозначение программного документа по ГОСТ 19.103-77: **RU.17701729.09.01-01 ТЗ 01–1** (числовой код вида документа по ГОСТ 19.101-77: **05**).

Вид программного изделия по ГОСТ 19.101-77, §2.2: **комплекс программ** — совокупность из восьми взаимодействующих программных компонентов, объединённых в единую систему и функционирующих в рамках общей программной среды.

1.3. Состав комплекса программ

Комплекс CSTATI.events включает восемь компонентов:

1. auth-manager — микросервис аутентификации и авторизации (Go 1.24);
2. events-service — микросервис управления мероприятиями (Go 1.24);
3. payment-service — микросервис обработки платежей (Go 1.24);
4. notifier — микросервис доставки уведомлений через Telegram (Go 1.24);
5. qr-service — микросервис генерации QR-кодов электронных билетов (Go 1.24);
6. tg-bot — Telegram-бот для взаимодействия с участниками (Python 3.10);
7. frontend — веб-интерфейс платформы (React 18.3);
8. nginx — обратный прокси и шлюз API (Nginx 1.25).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Настоящий документ распространяется на все восемь компонентов комплекса.

1.4. Краткая характеристика области применения

Платформа CSTATI.events предназначена для организации и проведения конференций, митапов, хакатонов, воркшопов и иных публичных мероприятий с ограниченным числом мест и обязательной онлайн-регистрацией.

Целевые области применения:

- образовательные учреждения (университеты, факультеты, кафедры), проводящие открытые лекции, семинары, защиты проектов;
- сообщества и клубы по интересам, организующие митапы и воркшопы;
- конференционные организаторы, которым требуется управляемая регистрация с разными ценовыми категориями, волнами регистрации и онлайн-оплатой.

Система автоматизирует полный жизненный цикл мероприятия: от публикации события и открытия регистрации до подтверждения участия, оплаты, выдачи электронного билета с QR-кодом и контроля прохода на мероприятие.

Категории пользователей системы:

- **Организаторы** — создают и настраивают мероприятия, управляют волнами регистрации, ценовыми категориями и промокодами, верифицируют проход участников на мероприятие по QR-коду электронного билета;
- **Участники** — просматривают мероприятия, регистрируются, производят онлайн-оплату через ЮKassa, получают электронный билет с уникальным QR-кодом в Telegram;
- **Администраторы** — пользователи с расширенными правами, имеющие доступ ко всем функциям организатора на любом мероприятии, а также к общесистемным административным инструментам платформы.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. ОСНОВАНИЯ ДЛЯ РАЗРАБОТКИ

2.1. Документы, на основании которых ведётся разработка

Разработка ведётся на основании следующих документов:

1. Устав федерального государственного автономного образовательного учреждения высшего образования «Национальный исследовательский университет «Высшая школа экономики»» (с изменениями) № 56 от 1.02.2016.
2. Приказ декана ФКН НИУ ВШЭ И. В. Аржанцева № 2.3-02/1112–04 от 11.12.2025 «Об утверждении тем, руководителей курсовых работ студентов образовательной программы Программная инженерия факультета компьютерных наук».

Организация, утвердившая документ-основание разработки: **Факультет компьютерных наук НИУ ВШЭ.**

Дата утверждения документа-основания: **11.12.2025.**

2.2. Наименование темы разработки

Наименование темы разработки — «Система регистрации и управления жизненным циклом мероприятий».

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. НАЗНАЧЕНИЕ РАЗРАБОТКИ

3.1. Функциональное назначение программы

Функциональное назначение определяет, какие задачи решает разработанное программное изделие, и какие операции оно выполняет.

Платформа CSTATI.events реализует автоматизацию управления жизненным циклом публичных мероприятий с ограниченным числом мест и предоставляет следующие функциональные возможности:

1. **Управление аутентификацией и авторизацией** — регистрация пользователей по адресу электронной почты, аутентификация с двухфакторной верификацией (одноразовый код через Telegram), управление сессиями, выдача и проверка токенов доступа (JWT), разграничение доступа по ролям (администратор, организатор, участник);
2. **Управление мероприятиями** — создание, редактирование и удаление событий; управление статусами мероприятий (черновик, запланировано, активно, завершено, отменено); настройка местоположения, даты, времени и описания события; публикация мероприятий в открытом доступе;
3. **Управление волнами регистрации** — определение нескольких волн регистрации для одного события; управление сроками открытия и закрытия волн; автоматическое управление статусами волн (запланирована, активна, приостановлена, завершена);
4. **Управление билетными категориями** — определение нескольких типов билетов (разные ценовые категории); настройка количества билетов в категории с атомарным резервированием мест; управление статусами билетов (активен, скрыт, распродан, архивирован);
5. **Управление промокодами** — создание кодов скидок с процентной скидкой; установка лимитов использования и сроков действия; атомарная активация промокодов с ограничением по частоте запросов;
6. **Регистрация участников** — приём заявок на участие с проверкой наличия свободных мест; применение промокодов и расчёт финальной стоимости;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

отправка заявки в очередь обработки; управление статусами регистрации (ожидание, подтверждена, отменена, истекла, неудачная, оплачена, пройден контроль);

7. **Обработка онлайн-платежей** — интеграция с платёжным шлюзом ЮKassa; создание платёжных ссылок; обработка вебхуков о статусе платежей; обеспечение идемпотентности операций; поддержка платных и бесплатных билетов;
8. **Генерация и доставка электронных билетов** — генерирование PNG-изображений QR-кодов размером 256×256 пиксельной; загрузка в объектное хранилище (MinIO); доставка билетов участникам через Telegram;
9. **Уведомления пользователей** — отправка сообщений через Telegram о подтверждении регистрации, ссылке на оплату, успешной оплате, отмене участия, кодах двухфакторной аутентификации; асинхронная обработка через очередь Kafka;
10. **Контроль прохода на мероприятие** — верификация участников по QR-коду электронного билета; предотвращение повторного входа; вывод информации о регистрации на контрольном пункте;
11. **Мониторинг работоспособности** — сбор метрик производительности (пропускная способность, латентность, частота ошибок); агрегирование и визуализация метрик в системе мониторинга.

3.2. Эксплуатационное назначение программы

Эксплуатационное назначение определяет условия использования программного изделия, окружение, в котором оно функционирует, и способы доступа конечных пользователей.

3.2.1. Архитектура развёртывания

Платформа CSTATI.events развёртывается как микросервисное приложение в контейнеризированной среде:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1. **Контейнеризация:** каждый микросервис упаковывается в Docker-контейнер; управление контейнерами осуществляется через Docker Compose в среде разработки и Kubernetes/Docker Swarm в продакшене;
2. **Сетевая архитектура:** все микросервисы взаимодействуют через внутреннюю сеть Docker; публичный доступ осуществляется через обратный прокси Nginx на портах 80 (HTTP) и 443 (HTTPS);
3. **Постоянное хранилище:** каждый микросервис имеет отдельную базу данных PostgreSQL; общее хранилище объектов MinIO для QR-кодов; кэш Redis для сессий и идиempотентности;
4. **Очереди сообщений:** асинхронная коммуникация между сервисами через Apache Kafka; топики: registrations, payments, notifications, qr-generation, registrations-dlg.

3.2.2. Каналы доступа пользователей

1. **Веб-браузер** — одностраничное приложение (React SPA), доступное по HTTPS на публичном домене. Пользователи взаимодействуют с системой через графический веб-интерфейс, поддерживающий как публичные страницы (каталог мероприятий, регистрация), так и защищённые разделы (личный кабинет, административная панель);
2. **Telegram-бот** — интерактивное приложение в мессенджере Telegram (webhook или long-polling). Участники используют бота для просмотра мероприятий, привязки аккаунта к платформе, получения уведомлений и электронных билетов;
3. **REST API** — стандартный HTTP API в формате OpenAPI 3.0, доступный для сторонних приложений и интеграций. Все запросы аутентифицируются через JWT-токены в заголовке Authorization: Bearer <token>.

3.2.3. Требования к окружению и обслуживанию

1. **Минимальные системные требования:** Linux-сервер (Ubuntu 20.04+) с минимум 4 ГБ оперативной памяти и 20 ГБ дискового пространства для хранения данных и логов;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. **Программное обеспечение:** Docker 20+, Docker Compose 2.0+; на сервере должны быть установлены компоненты Kafka, PostgreSQL, Redis (либо запущены в контейнерах);
3. **Сетевые требования:** наличие публичного IP-адреса или доменного имени для HTTPS; возможность терминирования TLS-соединений с помощью сертификатов Let's Encrypt;
4. **Обслуживание:** резервное копирование баз данных рекомендуется проводить ежедневно; логи сохраняются в папке `deploy/logs/` с ротацией по размеру (до 1 ГБ per file); мониторинг осуществляется через Prometheus + Grafana с сбором метрик каждые 30 секунд;
5. **Масштабируемость:** система рассчитана на одновременную работу от 10 до 100 организаторов и от 100 до 10 000 участников в зависимости от конфигурации ресурсов контейнеров.

3.2.4. Сценарии использования

Организатор мероприятия:

- входит в систему через веб-интерфейс или REST API;
- создаёт новое мероприятие с указанием даты, места и описания;
- настраивает волны регистрации и билетные категории;
- определяет промокоды и скидки;
- просматривает список зарегистрировавшихся участников в реальном времени;
- использует QR-сканер для контроля прохода участников на мероприятие.

Участник мероприятия:

- просматривает каталог доступных мероприятий через веб-браузер;
- регистрируется на интересующее событие через веб-форму;
- производит онлайн-оплату через платёжный шлюз ЮKassa;
- получает электронный билет с QR-кодом в Telegram;
- использует билет для прохода на мероприятие.

Администратор системы:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- управляет глобальными настройками платформы;
- мониторит производительность и доступность сервисов;
- контролирует логи ошибок и аномалии;
- выполняет резервное копирование и восстановление данных.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4. ТРЕБОВАНИЯ К ПРОГРАММЕ ИЛИ ПРОГРАММНОМУ ИЗДЕЛИЮ

4.1. Требования к функциональным характеристикам

4.1.1. Подсистема аутентификации и авторизации (auth-manager)

Подсистема реализуется как отдельный микросервис (auth-manager, Go 1.24) и должна обеспечивать (технические параметры — таблица 4.1):

1. **Регистрацию пользователей** (POST /api/v1/auth/sign_up): создание учётной записи. Обязательные поля: username (адрес электронной почты), password, telegram_username. Необязательные: first_name, last_name, settings.enable_newsletters. Запрос должен содержать заголовок User-Agent для дактилоскопии сессии. Пароль хранится в виде хэша Argon2id (16 МБ / 1 итерация / параллелизм 2). Поля telegram_username, first_name, last_name хранятся в БД в зашифрованном виде (шифрование на уровне приложения).
2. **Аутентификацию** (POST /api/v1/auth/sign_in): создание сессии и выдача пары токенов JWT после прохождения 2FA-верификации:
 - access-токен (TTL: 15 минут, поле typ = "access", издатель iss = "auth-manager");
 - refresh-токен (TTL: 720 часов / 30 суток, хранится в БД, может быть инвалидирован).
3. **Сессионную модель входа:**
 - GET /api/v1/auth/sessions/{sessionID} — получение статуса сессии; статус: AWAITING_TELEGRAM_2FA_CODE;
 - POST /api/v1/auth/sessions/{sessionID}/submit_2fa_code — подтверждение 6-значного кода; возвращает {access_token, refresh_token} при успехе;
 - POST /api/v1/auth/sessions/{sessionID}/resend_2fa_code — повторная отправка кода.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4. **Двухфакторную аутентификацию (2FA):** доставку 6-значного числового кода через Telegram-бот (публикация в Kafka-топик `notifications` → `notifier` → Telegram Bot API). Ограничения: не более 5 попыток ввода, не более 3 повторных отправок, интервал между отправками — не менее 10 с, TTL сессии — 5 минут.
5. **Обновление токена** (`POST /api/v1/auth/refresh_token`): выдача новой пары токенов по действующему `refresh`-токену.
6. **Управление пользователями** (требуется авторизация):
 - `GET /api/v1/auth/users/{userID}` — получение профиля: `id`, `username`, `telegram_username`, `first_name`, `last_name`, `settings`;
 - `GET /api/v1/auth/users` — постраничный список (параметры `page_size`, `page_offset`).
7. **Межсервисную аутентификацию** (gRPC, порт 50051):
 - `Auth.Authenticate(AccessToken)` — валидация JWT; возвращает `{UserID, IsAdmin}`;
 - `UserService.GetUser(UserID)` — получение профиля пользователя;
 - `TgService.BindUser(TgUserID, TgUsername)` — фиксация привязки Telegram-аккаунта;
 - `TgService.SendMessage(TgUsername, Text)` — отправка сообщения через `notifier`;
 - `PingService.Ping({})` — проверка доступности сервиса.
8. **Публикацию уведомлений:** отправку 2FA-кодов и системных событий в Kafka-топик `notifications`.

Таблица 4.1 — Параметры подсистемы аутентификации

Параметр	Значение
HTTP-порт	8001
gRPC-порт	50051

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Параметр	Значение
База данных	auth_db (PostgreSQL 16)
TTL access-токена	15 минут
TTL refresh-токена	720 часов (30 суток)
Длина кода 2FA	6 цифр
TTL кода 2FA	5 минут
Макс. попыток 2FA	5
Макс. повторных отправок	3
Интервал повторной отправки	10 секунд
Алгоритм хэширования	Argon2id (16 МБ / 1 итерация / параллелизм 2)
Prefix пути API	/api/v1/auth/

Нефункциональные требования к auth-manager:

- Авторизация и безопасность:** все приватные HTTP-эндпоинты auth-manager, кроме sign_up, sign_in, refresh_token, должны требовать валидный access JWT. Подпись JWT должна проверяться по алгоритму HMAC-SHA256, поля exp, iat, iss, typ должны валидироваться строго.
- Rate-limit:** для маршрутов /api/v1/auth/* должен применяться лимит Nginx 50 запросов/сек на IP (зона auth_limit) с burst=20 и nodelay; при превышении возвращается 429.
- Производительность:** p95 времени ответа sign_in и refresh_token не более 250 мс при нагрузке до 50 RPS на экземпляр; p99 не более 500 мс.
- Надежность 2FA:** отправка кода не должна приводить к блокировке основного потока аутентификации; при недоступности Telegram код должен переправляться через retry-контур notifier без потери сессии.
- Аудит и трассировка:** операции входа, неуспешной проверки кода и обновления токена должны логироваться с trace_id, user_id/session_id, IP и user-agent без записи чувствительных данных.

4.1.2. Подсистема управления мероприятиями (events-service)

Подсистема реализуется как два компонента в рамках одного модуля (events-service, Go 1.24); технические параметры — таблица 4.2.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Компонент «API-сервер» (cmd/api). Предоставляет HTTP REST API (Echo v4) и должен обеспечивать:

- 1. Управление мероприятиями (Events CRUD):** поля: id, owner_id, name (5–255 символов), description (до 2000 символов), slug (уникальный URL-идентификатор, /^[a-z0-9-]+\$/, 1–100 символов), start_date, end_date, location, status (draft~| scheduled~| live~| completed~| cancelled), allowed_payment_types (online и / или transaction). Список мероприятий: параметры skip (по умолчанию 0), limit (по умолчанию 50), status. Эндпоинты: GET /api/v1/events, POST /api/v1/events, GET /api/v1/events/{slug}, PATCH /api/v1/events/{event_id}.
- 2. Управление волнами регистрации (Waves):** поля при создании: waveNumber (целое ≥ 1), name, description, startAt, endAt, status, autoClose (флаг автозакрытия по расписанию). Статусный автомат: scheduled $\rightarrow$ active $\rightarrow$ paused / sold_out $\rightarrow$ closed / expired. Эндпоинты: GET /api/v1/events/{event_id}/waves, POST /api/v1/events/{event_id}/waves, GET /api/v1/events/{event_id}/waves/{wave_id}, PATCH /api/v1/events/{event_id}/waves/{wave_id}, POST /api/v1/events/{event_id}/waves/{wave_id}/close.
- 3. Управление билетами (Tickets):** билеты привязаны к волне. Поля: name, description, price (вещественное число, минимальная единица — копейка), quantity (>0), status (draft~| active~| hidden~| sold_out~| archived). Атомарное резервирование мест с оптимистичной блокировкой (version-поле). Эндпоинты: GET /api/v1/events/{event_id}/waves/{wave_id}/tickets, POST /api/v1/events/{event_id}/waves/{wave_id}/tickets, GET /api/v1/tickets/{ticket_id}, PATCH /api/v1/tickets/{ticket_id}.
- 4. Промокоды (Promocodes):** привязаны к волне. Поля: code, sale_percent (хранится как доля, 0,15 = 15 %), count (лимит использования), valid_from, valid_to. Атомарная оптимистичная блокировка. Эндпоинты: GET /api/v1/events/{event_id}/waves/{wave_id}/promocodes, POST /api/v1/events/{event_id}/waves/{wave_id}/

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

promocodes, GET /api/v1/promocodes/{id}, PATCH /api/v1/promocodes/{id}, DELETE /api/v1/promocodes/{id}, POST /api/v1/promocodes/{code}/activate.

Активация (POST .../activate): тело {wave_id: UUID}; возвращает {promocode_lock_id, discount_percent, expires_at} (блокировка действительна 30 мин; лимит 5 попыток/мин с одного IP).

5. **Регистрацию участников** (POST /api/v1/registrations): заголовок Idempotency-Key: <UUID> обязателен, кэш в Redis 24 ч. Обязательные поля тела: fio (5–255 символов), email (RFC 5322), tg (5–100 символов, без @), ticket_id (UUID), payment_method (online~| transaction). Необязательные: user_id (UUID), promocode_lock_id (UUID, выданный эндпоинтом активации). Статусы регистрации: pending~| confirmed~| cancelled~| expired~| failed~| paid~| checked_in. Срок ожидания оплаты — 72 ч с момента создания.
6. **Check-in**: GET /api/v1/checkin/{registration_id} — получение данных регистрации для экрана проверки; POST /api/v1/checkin/{registration_id} — подтверждение прохода (переводит status → checked_in; повторный check-in возвращает ошибку 409). Требуется права администратора / владельца мероприятия.
7. **Просмотр списка гостей** (GET /api/v1/events/{event_id}/guests): страничный список регистраций с параметрами page (1), limit (1–100, по умолчанию 20). Требуется права владельца мероприятия.
8. **Метрики** (GET /metrics): эндпоинт в формате Prometheus.

Компонент «Worker» (cmd/worker). Обработывает события из Kafka и реализует три независимых воркера:

1. **RegistrationWorker**: консьюмит топик registrations (consumer group event-service-worker); в рамках единой PostgreSQL-транзакции резервирует места и публикует PaymentRequestMessage в топик payments. При ошибке сообщение попадает в DLQ после 5 попыток.
2. **PaymentWorker**: консьюмит топик payments; переводит статус регистрации (pending → confirmed или expired); при успехе подтверждает продажу

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

билета (reserved → sold) и публикует уведомление в notifications; при отказе освобождает резерв и откатывает использование промокода.

3. **ExpirationWorker:** запускается каждые 30 секунд; выбирает просроченные регистрации (pending) пакетами по 100 записей; переводит в expired и освобождает зарезервированные места.

Таблица 4.2 — Параметры подсистемы управления мероприятиями

Параметр	Значение
HTTP-порт (API)	8000
База данных	event_service (PostgreSQL 16)
Кэш	Redis 7 (pool: 100 соединений)
Consumer Group (Worker)	event-service-worker
Параллелизм воркеров	4 горутин
Период ExpirationWorker	30 секунд
Размер пакета ExpirationWorker	100 записей
TTL резерва регистрации	72 часа
TTL резерва промокода	30 минут
TTL кэша идемпотентности	24 часа
Попытки retry (до DLQ)	5 (экспоненциальная задержка: 1 с, ×2, макс. 30 с)
Circuit breaker (к auth)	порог отказов 20 %, мин. запросов 20
Макс. открытых соединений с БД	50

Нефункциональные требования к events-service:

1. **Авторизация:** операции изменения состояния (POST/PATCH/DELETE, check-in, админские списки гостей) должны выполняться только после gRPC-проверки Auth.Authenticate; при недоступности auth-manager и открытом circuit breaker возвращается 503.
2. **Rate-limit:** для эндпоинтов /api/v1/events* применяется лимит 300 запросов/сек (зона events_limit), для /api/v1/registrations, /api/v1/tickets, /api/v1/promocodes, /api/v1/checkin — 100 запросов/сек (зона api_limit).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. **Идемпотентность:** POST /api/v1/registrations обязан возвращать идентичный ответ при повторном Idempotency-Key в течение TTL 24 ч и не создавать повторных резервов мест.
4. **Временные характеристики:** создание регистрации (HTTP + запись в БД + постановка сообщения) — p95 не более 800 мс; получение карточки мероприятия — p95 не более 150 мс; выдача списка гостей (страница 20 записей) — p95 не более 300 мс.
5. **Отказоустойчивость воркеров:** при падении downstream-сервисов обработки Kafka должны применять retry с экспоненциальной задержкой; после 5 неудач сообщение обязано переводиться в DLQ с сохранением исходного payload и причины ошибки.
6. **Консистентность:** операции резервирования мест/списания/возврата должны выполняться в транзакции с изоляцией не ниже Read Committed и проверкой version для оптимистичной блокировки.

4.1.3. Подсистема обработки платежей (payment-service)

Подсистема реализуется как микросервис (payment-service, Go 1.24) и должна обеспечивать (технические параметры — таблица 4.3):

1. **Интеграцию с ЮKassa:** создание платежей через REST API ЮKassa (POST `https://api.yookassa.ru/v3/payments`), аутентификация HTTP Basic Auth (Shop ID / Secret Key). Таймаут запроса — 10 секунд.
2. **Обработку платных билетов:**
 - получение PaymentRequestMessage из Kafka-топика payments;
 - создание платежа с типом подтверждения redirect;
 - отправка ссылки для оплаты через топик notifications;
 - обработка webhook-уведомлений от ЮKassa (POST /api/v1/payment/webhook);
 - редирект пользователя на страницу результата (GET /api/v1/payment/success?registration_id={id}).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. **Обработку бесплатных билетов:** при нулевой стоимости — немедленная публикация статуса `confirmed` в топик `payments` без обращения к ЮKassa.
4. **Начисление комиссии:** настраиваемый процент (по умолчанию 5 %) прибавляется к стоимости билета перед созданием платежа.
5. **Идемпотентность платежей:** передача `registration_id` в качестве ключа идемпотентности при создании платежа в ЮKassa; повторный запрос возвращает существующий платёж.
6. **Публикацию статусов:** отправку результатов в топики `notifications` (уведомление пользователю) и `payments` (обратный статус для `events-worker`).

Таблица 4.3 — Параметры подсистемы обработки платежей

Параметр	Значение
HTTP-порт	8004
База данных	<code>payment_db</code> (PostgreSQL 16)
Платёжный шлюз	ЮKassa (<code>yookassa.ru</code>), API v3
Комиссия (по умолчанию)	5 %
Таймаут API ЮKassa	10 секунд
Consumer Group	<code>payment-service</code>
Валюта	RUB (суммы в копейках)
Webhook-эндпоинт	<code>POST /api/v1/payment/webhook</code>
Success-эндпоинт	<code>GET /api/v1/payment/success</code>

Нефункциональные требования к `payment-service`:

1. **Надежность внешней интеграции:** при сетевых ошибках ЮKassa должны выполняться повторные попытки запроса с `backoff`; суммарное время внешнего вызова не должно превышать 30 секунд на операцию.
2. **Безопасность:** `webhook` от ЮKassa должен валидироваться по подписи и/или набору обязательных полей события; запросы без валидной подписи отклоняются кодом 401/403.
3. **Идемпотентность:** повторная обработка `webhook` одного платежа не должна изменять состояние регистрации более одного раза.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4. **Производительность:** обработка webhook (включая запись в БД и публикацию в Kafka) — p95 не более 500 мс.
5. **Rate-limit:** для публичных payment-эндпоинтов применяется лимит 100 запросов/сек (зона api_limit) с отдельным контролем burst для webhook.

4.1.4. Подсистема уведомлений (notifier)

Подсистема реализуется как микросервис (notifier, Go 1.24) и должна обеспечивать (технические параметры — таблица 4.4):

1. **Консьюминг уведомлений:** чтение событий из Kafka-топика notifications (consumer group notifier-service, смещение OffsetOldest). Схема сообщения: {type, tg_username, text, tg_user_id?, email?, image_url?}.
2. **Доставку через Telegram:** поиск chat_id по tg_username в таблице tg_bindings, отправка сообщения через Telegram Bot API (ParseMode: HTML). Поддерживаются следующие типы уведомлений:
 - registration_pending — подтверждение создания заявки (ссылка на оплату);
 - registration_success — регистрация успешно завершена;
 - registration_failed — ошибка при обработке регистрации;
 - payment_created — ссылка на оплату создана;
 - payment_succeeded — оплата принята;
 - payment_canceled — оплата отменена / не прошла;
 - payment_success — платёж подтверждён (дублирует для пользователя);
 - payment_failed — платёж отклонён;
 - qr_code — электронный билет с QR-кодом (содержит поле image_url);
 - bind_user — специальный тип: не отправляет сообщение, а создаёт / обновляет запись в tg_bindings.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. **Привязку аккаунтов:** при получении сообщения типа `bind_user` — `upsert` в таблицу `tg_bindings(tg_username PK, tg_user_id BIGINT)` БД `notifier_db`.
4. **gRPC-интерфейс** (порт 50052):
 - `TgService.SendMessage(TgUsername, Text)` — прямая отправка сообщения;
 - `TgService.BindUser(TgUserID, TgUsername)` — синхронная регистрация привязки;
 - `PingService.Ping({})` — проверка доступности.
5. **HTTP-интерфейс:** `GET /api/notifier/health` — проверка работоспособности; `GET /metrics` — метрики Prometheus.

Таблица 4.4 — Параметры подсистемы уведомлений

Параметр	Значение
HTTP-порт	8002
gRPC-порт	50052
База данных	<code>notifier_db</code> (PostgreSQL 16)
Consumer Group	<code>notifier-service</code> (OffsetOldest)
Канал доставки	Telegram Bot API (ParseMode: HTML)
Типов уведомлений	10
Таблица привязок	<code>tg_bindings(tg_username PK, tg_user_id BIGINT)</code>

Нефункциональные требования к `notifier`:

1. **Гарантия доставки:** при временной недоступности Telegram Bot API сообщение не теряется, а переводится в `retry`-контур; при исчерпании `retry` формируется событие ошибки для наблюдаемости.
2. **Производительность:** подтверждение Kafka-offset допускается только после фактического ответа Telegram API; среднее время обработки одного сообщения не должно превышать 300 мс.
3. **Надежность привязок:** операция `bind_user` должна быть идемпотентной (`upsert`) и выдерживать повторную доставку Kafka-сообщений без дублей.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4. **Безопасность:** gRPC-методы отправки сообщений должны быть доступны только доверенным сервисам внутреннего контура; прямой внешний доступ к gRPC-порту запрещен firewall-правилами.

4.1.5. Подсистема Telegram-бота (tg-bot)

Подсистема реализуется как микросервис (tg_bot, Python 3.10+, aiogram 3) и должна обеспечивать:

1. **Пользовательский интерфейс в Telegram:** обработку команд:

- /start — если у пользователя есть username в Telegram, публикует сообщение {type: "bind_user", tg_username, tg_user_id} в Kafka-топик notifications (группа notifier-service); при отсутствии username — возвращает ошибку;
- /help — вывод справки по доступным командам;
- **любые другие сообщения** — возвращает реакцию 🤖.

2. **Два режима работы,** переключаемых через конфигурацию:

- **Webhook-режим** (продакшн): Telegram отправляет обновления на HTTP-эндпоинт POST /api/tg_bot/webhook; рекомендован для production;
- **Long-polling-режим:** бот периодически опрашивает серверы Telegram; используется для локальной разработки.

3. **Привязку аккаунта:** при команде /start публикует событие bind_user в Kafka; сервис notifier обрабатывает событие и сохраняет пару (tg_username, tg_user_id) в таблице tg_bindings.

4. **HTTP-сервер** на порту 8080: приём webhook-запросов от Telegram.

Нефункциональные требования к tg-bot:

1. **Устойчивость к флудам:** входящие обновления Telegram обрабатываются асинхронно очередью обработчиков; бот должен сохранять работоспособность при burst до 30 сообщений/сек.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. **Безопасность webhook:** в webhook-режиме должен проверяться секретный токен Telegram и допустимый source IP (при возможности платформы).
3. **Деградация:** при недоступности Kafka бот должен возвращать пользователю понятное сообщение о временной ошибке и не завершаться аварийно.

4.1.6. Подсистема генерации QR-кодов (qr-service)

Подсистема реализуется как микросервис (qr-service, Go 1.24) и должна обеспечивать:

1. **Асинхронное потребление заданий:** консьюминг топика qr-generation (consumer group qr-service-group); каждое сообщение содержит: registration_id, event_id, ticket_id, user_id, tg_username, event_title, ticket_title.
2. **Генерацию QR-кодов:** создание PNG-изображений разрешением **512×512 пикселей**, кодирующих URL вида {checkin.base_url}/checkin/{registration_id} (готовая ссылка для сканирования на входе).
3. **Хранение в объектном хранилище:** загрузку готового PNG в MinIO/S3 (bucket qr-codes, ключ qr/{registration_id}.png); формирование публичного URL изображения.
4. **Публикацию результата:** отправку в Kafka-топик notifications сообщения типа qr_code с полями tg_username, image_url, text («Ваш билет на мероприятие «{EventTitle}»\nТип билета: {TicketTitle}\nПокажите QR-код на входе.»).
5. **HTTP-эндпоинт:** GET /api/qr/health — проверка работоспособности.

Нефункциональные требования к qr-service:

1. **Производительность:** время генерации и сохранения одного QR-кода (без ожидания Kafka) — не более 400 мс в среднем и 800 мс p95.
2. **Надежность хранения:** при успешной генерации объект обязательно должен быть записан в S3/MinIO до отправки уведомления; нарушение порядка операций не допускается.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. **Идемпотентность:** повторная обработка того же `registration_id` должна перезаписывать файл по тому же ключу `qr/{registration_id}.png` без создания дубликатов.
4. **Совместимость:** сгенерированный PNG должен быть читаем стандартными сканерами QR-кодов на Android и iOS.

4.1.7. Веб-интерфейс (frontend)

Общие сведения:

1. одностраничное приложение; взаимодействие с API по HTTPS;
2. в качестве базы клиента используются React, Vite, клиент маршрутизации;
3. оформление полей, проверок и сообщений — на русском языке, без вывода внутренних кодов ошибок сервисов пользователю.

Шапка и подвал (общие для всех экранов в основной вёрстке):

1. **шапка:** логотип и наименование «cstat»; ссылки «Каталог» и «Мои регистрации»; при праве администратора — «Админ панель»; у вошедшего пользователя — «Профиль» и выход; сокращённое меню на узких экранах;
2. **подвал:** наименование и строка с годом в копирайте;
3. **отдельно:** экран проверки билета (check-in) отображается без общей шапки и подвала, полноэкранный.

Правила доступа к маршрутам:

1. гостю недоступны «Мои регистрации» и «Профиль» без входа — при обращении выполняется переход на экран входа;
2. панель администратора — только у пользователя с правом администратора; иначе — переход в каталог мероприятий;
3. у вошедшего пользователя при открытии экранов «Вход» и «Регистрация» выполняется перенаправление в зависимости от роли (каталог или панель администратора) либо на двухфакторную проверку, если сессия входа ещё не завершена;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4. корневой адрес приложения ведёт: при незавершённой проверке кода — на двухфакторную проверку; при отсутствии сессии — в каталог; при вошедшем администраторе — к списку мероприятий в панели, иначе — в каталог.

Таблица 4.4а — Адреса и ограничение доступа

Адрес	Название экрана	Кто открывает
/	переход в раздел по умолчанию	все, кроме отдельных правил в списке выше
/login	вход в систему	все (не вошёл)
/register	регистрация учётной записи	все (не вошёл)
/verification	двухфакторная проверка (код в Telegram)	все; без данных незавершённого входа — переход к экрану входа
/events	каталог мероприятий	все
/events/:slug	карточка мероприятия	все
/events/:slug/register	регистрация на мероприятие	все
/events/:slug/register/success	сообщение после отправки заявки	все
/my-registrations	мои регистрации	вошедший пользователь
/profile	профиль	вошедший пользователь
/checkin/:id	проверка билета (по идентификатору в адресе)	вошедший администратор; вне общей шапки
/admin/events	список мероприятий (панель)	администратор
/admin/events/create	создание мероприятия	администратор

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Адрес	Название экрана	Кто открывает
/admin/events/:eventId	карточка мероприятия (панель)	администратор
/admin/events/:eventId/registrations	регистрации по мероприятию	администратор

Экраны:

1. каталог мероприятий (/events)

- **содержимое:** заголовок; поиск по названию; фильтр по статусу; карточки: статус, название, краткое описание, даты, место, кнопка «Подробнее»; при отсутствии соответствий — подсказка и сброс фильтра; при пустом списке — пояснение; индикатор загрузки и сообщение об ошибке;
- **переходы:** клик по карточке или «Подробнее» — карточка выбранного мероприятия; черновики в каталоге не показываются;

2. карточка мероприятия (публичная) (/events/:slug)

- **содержимое:** ссылка «К каталогу»; заголовок; признак «Открыта регистрация»/«Закрыта»; даты, место, полное описание; блок волны и типов билетов: наименование, цена, остаток, кнопка «Зарегистрироваться»; полный перечень волн в отдельном списке **не выводится**;
- **переходы:** «К каталогу» — каталог; «Зарегистрироваться» — форма регистрации (недоступна при закрытой продаже или нулевом остатке);

3. регистрация на мероприятие (/events/:slug/register)

- **содержимое:** ссылка к карточке мероприятия; заголовок и пояснение; поля: Ф.И. О., адрес электронной почты, идентификатор в Telegram, выбор билета, способ оплаты (карта онлайн / перевод), промокод и кнопка «Применить»; блок итоговой суммы; сообщения об ошибках; индикатор отправки;
- **переходы:** «Вернуться к мероприятию» — публичная карточка; после успешной отправки — экран благодарности; ведущий символ «@» в поле Telegram при отправке снимается;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4. после отправки заявки (/.../register/success)

- **содержимое:** поздравление, ссылки в чат бота и в каталог, краткие шаги «что далее»;
- **переходы:** в каталог;

5. вход (/login)

- **содержимое:** пояснение про Telegram-бота; поля «Имя пользователя» и «Пароль»; кнопка входа; сообщение об ошибке; ссылка на регистрацию;
- **переходы:** «Зарегистрироваться» — регистрация; при успешной первой стадии — двухфакторная проверка;

6. регистрация учётной записи (/register)

- **содержимое:** пояснение про бота; поля: имя пользователя, Telegram, пароль, подтверждение пароля, отметка «Активировал бота»; кнопка отправки; сообщение об ошибке; ссылка на вход;
- **переходы:** «Войти» — вход; при успехе — двухфакторная проверка; данные сессии незавершённого входа в том числе в долговременном хранилище браузера;

7. двухфакторная проверка (/verification)

- **содержимое:** пояснение; шесть ячеек для кода; кнопки «Подтвердить» и «Отправить код» (повтор с периодом ожидания 10 с); сообщения об ошибке; ссылка «Назад»; ограничение числа неверных попыток и числа повторов в интерфейсе **не отображается**;
- **переходы:** «Назад» — регистрация или вход; при успехе — каталог или панель администратора в зависимости от роли;

8. мои регистрации (/my-registrations)

- **содержимое:** предупреждение, что сведения о регистрациях доступны в Telegram-боте;
- **переходы:** через шапку;

9. профиль (/profile)

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- **содержимое:** имя пользователя, при наличии — метка «Администратор», строка с ролью;
- **переходы:** через шапку;

10. проверка билета (организатор) (/checkin/...)

- **содержимое:** при необходимости — предложение войти; сведения о регистрации, статусе, билете, сумме; кнопка «Отметить прибытие» при допустимом статусе; иначе — пояснение; ссылка «На главную»;
- **переходы:** «Войти» — вход; «На главную» — переход в раздел по умолчанию;

11. панель: список мероприятий (/admin/events)

- **содержимое:** заголовок; поиск; фильтр по статусу; кнопка «Создать мероприятие»; карточки; пустой список с подсказкой; загрузка и ошибка;
- **переходы:** «Создать мероприятие» — форма создания; клик по карточке — карточка мероприятия в панели;

12. панель: создание мероприятия (/admin/events/create)

- **содержимое:** ссылка к списку; поля: название, адрес (slug) латиницей, описание, место, дата и время начала и окончания; кнопки «Создать» и «Отмена»; сообщения об ошибках;
- **переходы:** «Отмена» — список; при успехе — карточка созданного мероприятия;

13. панель: карточка мероприятия (/admin/events/:eventId)

- **содержимое:** ссылка к списку; заголовок, статусы, даты, место; вкладки «Мероприятие» (текст-памятка и редактирование полей, смена статуса в допустимых направлениях), «Волны» (волны, билеты, промокоды), «Регистрации» (гости); при необходимости смена подраздела через параметр ?tab= в адресе;
- **переходы:** к списку; отдельный адрес с суффиксом .../registrations ведёт к тому же разделу гостей;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

14. панель: регистрации (отдельный адрес) (/admin/events/:eventId/registrations)

- **содержимое:** ссылка «К событию»; тот же список гостей, что и на вкладке: поиск по Ф.И. О., почте, Telegram; фильтр по волне; пагинация (по 20 записей) через ?page=; кнопка «Обновить»;
- **переходы:** «К событию» — карточка мероприятия в панели.

Таблица 4.46 — Клиентская проверка вводимых данных

Экран	Поле	Правила
регистрация учётной записи	имя пользователя	обязательно; не короче 3 символов
регистрация учётной записи	Telegram	обязательно; от 3 до 25 символов
регистрация учётной записи	пароль	обязательно; не короче 8 символов
регистрация учётной записи	подтверждение пароля	совпадение с паролем
регистрация учётной записи	чекбокс про бота	должен быть отмечен
двухфакторная проверка	код	6 цифр, обязательно
создание мероприятия	название, адрес, даты, описание, место	длины и логичность дат по экрану создания
регистрация на мероприятие	Ф.И. О., почта, Telegram, билет, оплата	см. форму: длины, формат почты, выбор обязателен
вход	логин и пароль	только ответ сервера при ошибке; собственных правил ввода нет

Требования к пользовательскому интерфейсу:

1. адаптивный дизайн: мобильные (ширина не менее 320 px), планшет (от 768 px), настольный (от 1024 px);
2. индикаторы ожидания при запросах;
3. сообщения об ошибках на русском языке, в понятном для пользователя виде.

Нефункциональные требования к клиенту:

1. **Авторизация и доступ к экранам:** разграничение маршрутов встроено в клиентское приложение; при отказе в доступе по истёкшему сроку действия токена — запрос на его обновление; при неуспехе обновления — выход из системы и очистка данных сессии.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. **Хранение токенов и идентификаторов сессии:** пары токенов доступа и обновления — в долговременном хранилище браузера; в строке адреса не передавать; в журнал (консоль) не выводить; идентификатор сессии двухфакторной проверки — в том же долговременном хранилище, а не в отдельной сессии вкладки.
3. **Производительность отображения:** показатели быстрой первой отрисовки (FCP) на столе в промышленной сборке — не более 2,5 с при стандартном сетевом профиле Fast 3G/4G; фактические значения фиксировать при приёмо-сдаточных испытаниях.
4. **Устойчивость к сбоям:** при ошибках сети и ответов сервера — отображать сообщения в интерфейсе, на русском языке; обеспечить, чтобы сценарий не зависал; где предусмотрено (например, список гостей) — кнопка обновления данных; требовать явного действия «повторить» на каждом экране **не обязательно**.

4.1.8. Подсистема мониторинга и наблюдаемости

Подсистема реализуется на базе набора инструментов с открытым исходным кодом и должна обеспечивать:

1. **Сбор метрик:** каждый Go-сервис (auth-manager, events-api, events-worker, payment-service, notifier, qr-service) экспортирует метрики в формате Prometheus на эндпоинте GET /metrics.
2. **Сервер метрик:** Prometheus — скрейпинг всех сервисов по расписанию, хранение временных рядов, предоставление PromQL API.
3. **Визуализацию:** Grafana 10.2.2 с источником данных Prometheus и дашбордами для каждого сервиса. URL в продакшне: <https://cstati.com/monitoring/grafana/>. Стандартные метрики: HTTP-запросы (latency, RPS, коды ответов), ошибки, Kafka consumer lag.
4. **Управление топиками Kafka:** Kafka UI (внутренний порт 9090) — веб-интерфейс для просмотра топиков, групп потребителей и lag-метрик.
5. **Управление кэшем Redis:** Redis Commander — веб-интерфейс для просмотра и редактирования ключей Redis в реальном времени.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

6. **Распределённую трассировку:** Jaeger All-in-one — сбор и визуализация span-трейсов по протоколу OpenTelemetry. Каждый Go-сервис инструментирован для отправки трейсов (HTTP-запросы, Kafka-операции, SQL-запросы) с передачей контекста через заголовки `traceparent`.

4.1.9. Инфраструктурный и эксплуатационный контур

Платформа должна разворачиваться как единый производственный контур в среде виртуальных машин с использованием контейнеризации, обратного прокси и доменной маршрутизации.

1. **Docker и контейнерная поставка:** каждый сервис (`auth-manager`, `events-service API/worker`, `payment-service`, `notifier`, `qr-service`, `tg-bot`, `frontend`) должен собираться в отдельный контейнерный образ с фиксируемой версией; запуск — через Docker Compose с декларативным описанием сетей, переменных окружения, `healthcheck` и томов.
2. **Kafka-контур:** в инфраструктуре должен быть развёрнут брокер Kafka с настроенными топиками `registrations`, `payments`, `notifications`, `qr-generation`, а также DLQ-топиками для неуспешной обработки. Консьюмер-группы сервисов должны быть изолированы и наблюдаемы по лаг-метрикам.
3. **PostgreSQL-контур:** должны поддерживаться отдельные базы данных микросервисов (`auth_db`, `event_service`, `payment_db`, `notifier_db`) и миграции схемы. Все сервисы обязаны работать с явными лимитами пула соединений и таймаутами SQL-операций.
4. **Redis-контур:** Redis используется для идемпотентности, временных блокировок, ограничений повторных действий и кэширования промежуточных результатов. Для всех ключей должны быть заданы TTL и префиксы `namespace`.
5. **S3-контур:** хранение артефактов QR-кодов должно выполняться в S3-совместимом объектном хранилище (MinIO/S3) с формированием воспроизводимого пути хранения и публичного URL доступа по политике `bucket`.
6. **Nginx и внешний периметр:** внешний доступ к системе должен обеспечиваться через Nginx с TLS-терминацией, маршрутизацией по `path-prefix` к

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

внутренним сервисам, rate-limit по IP и защитой служебных маршрутов (Swagger, Grafana и др.) через аутентификацию.

Нефункциональные требования к nginx:

1. **Доступность:** Nginx должен работать в режиме reverse проху с целевой доступностью не ниже 99,9 % в месяц.
2. **Безопасность:** поддержка только TLS 1.2+; weak cipher suites должны быть отключены.
3. **Rate-limit и anti-abuse:** превышение лимита должно возвращать код 429 с единым телом ошибки; для служебных URL должна применяться базовая аутентификация и ограничение по IP (при наличии allowlist).
4. **Домен и DNS:** эксплуатационный контур должен быть опубликован на домене cstat.i.com. DNS-записи (A/AAAA/CNAME/TXT при необходимости валидации) должны быть настроены так, чтобы обеспечивать доступ к основному сайту и сервисным поддоменам.
5. **Аренда и подготовка ВМ:** в рамках работ требуется аренда и подготовка ВМ (CPU/RAM/диск/сеть), установка Docker-окружения, настройка системного firewall, ротации логов и базовых мер hardening.
6. **CI/CD-контур:** pipeline должен обеспечивать автоматическую проверку кода, генерацию артефактов (включая `oapi-codegen/sqlc`), сборку контейнерных образов, прогон тестов, публикацию образов и контролируемый деплой в целевую ВМ-инфраструктуру.

4.2. Требования к надёжности

1. Коэффициент доступности системы — не менее 99,5 % в рабочее время.
2. Все критические операции с билетами (резервирование, подтверждение, освобождение) выполняются атомарно в рамках транзакций PostgreSQL с возможностью полного отката.
3. Для защиты от потери сообщений применяется Dead Letter Queue (DLQ): топик `registrations-dlq`. Сообщения, не обработанные после 5 попыток

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

с экспоненциальной задержкой (начальная — 1 с, максимальная — 30 с, множитель — 2,0), направляются в DLQ.

4. Идемпотентность регистраций обеспечивается заголовком `Idempotency-Key` с кэшированием в Redis (TTL 24 ч): повторный запрос с тем же ключом возвращает сохранённый ответ без создания дубликата.
5. Идемпотентность платежей обеспечивается передачей `registration_id` в качестве ключа идемпотентности API ЮKassa.
6. Redis-кэш снижает нагрузку на базу данных при пиковых запросах (пул соединений — 100).
7. Все сервисы выполняют `graceful shutdown` при получении `SIGTERM`: дожидаются завершения текущих запросов/сообщений, закрывают соединения с базами данных, Kafka и Redis.
8. Все Docker-контейнеры снабжаются `healthcheck`-проверками; при неуспешном прохождении — перезапускаются политикой `restart: always`.

4.3. Условия эксплуатации

4.3.1. Климатические условия эксплуатации

Заданные характеристики должны обеспечиваться при следующих климатических условиях:

1. температура окружающего воздуха от -25°C до $+35^{\circ}\text{C}$;
2. относительная влажность воздуха от 40 % до 80 % (без конденсата);
3. атмосферное давление от 700 до 820 мм рт. ст.

4.3.2. Требования к видам обслуживания

Система развёртывается на облачном сервере под управлением Linux (Ubuntu 20.04 LTS или новее) с использованием Docker Compose. Предусматривается непрерывная круглосуточная (24/7) работа системы. Техническое обслуживание (обновление, резервное копирование) производится во время плановых технических

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

окон. На устройстве, на котором планируется эксплуатация Системы, должны отсутствовать вирусы и сторонние программы, конфликтующие с работой сервисов.

4.3.3. Количество и квалификация пользователей

Для работы Системы требуется не менее одного организатора мероприятия в роли администратора. Администратор должен обладать базовыми навыками работы с веб-интерфейсами и мессенджером Telegram. Обслуживание серверной части требует квалификации инженера DevOps: знание Linux, Docker Compose, основ сетевого администрирования.

4.4. Требования к составу и параметрам технических средств

Минимальные требования к серверу:

Таблица 4.6 — Минимальные требования к серверному оборудованию

Компонент	Минимальные требования
Процессор	4 физических ядра, 64-разрядная архитектура x86-64
Оперативная память	8 ГБ RAM
Постоянное хранилище	SSD 50 ГБ (рекомендуется 100 ГБ)
Пропускная способность сети	100 Мбит/с
Операционная система	Linux (Ubuntu 20.04 LTS / Debian 11 или новее)
Docker Engine	версия 24.0 и выше
Docker Compose	версия 2.0 (Compose V2) и выше
Открытые порты	80/TCP (HTTP), 443/TCP (HTTPS)

Лимиты ресурсов Docker-контейнеров:

Таблица 4.7 — Лимиты ресурсов контейнеров

Сервис	CPU (лимит)	RAM (лимит)
postgres	2 ядра	2 ГБ
kafka	2 ядра	2 ГБ
redis	1 ядро	512 МБ
events-api	1 ядро	512 МБ
events-worker	1 ядро	512 МБ
auth-manager	0,5 ядра	256 МБ

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Сервис	CPU (лимит)	RAM (лимит)
notifier	0,5 ядра	256 МБ
payment-service	0,5 ядра	256 МБ
tg-bot	0,5 ядра	256 МБ
qr-service	0,5 ядра	256 МБ
grafana	0,5 ядра	512 МБ
minio	0,5 ядра	512 МБ
Итого (резерв)	≈ 10 ядер	≈ 8 ГБ

4.5. Требования к информационной и программной совместимости

- Операционная система сервера:** Linux, ядро 5.4 и выше (64-бит, x86-64).
- Контейнеризация:** Docker Engine 24.0+, Docker Compose 2.0+; образы базируются на alpine-вариантах для минимизации размера.
- Языки и среды выполнения:**
 - Go 1.24.0 (auth-manager, events-service, notifier, payment-service, qr-service);
 - Python 3.10+ (tg-bot, aiogram 3);
 - Node.js 20 LTS (сборка фронтенда; не требуется в production).
- Базы данных:** PostgreSQL 16 (alpine); клиентская библиотека pgx v5; миграции — goose.
- Брокер сообщений:** Apache Kafka 7.4.0 (Confluent), ZooKeeper 3.8.2.
- Объектное хранилище:** MinIO (S3-совместимый API, AWS SDK v2).
- Обратный прокси:** Nginx 1.25 (alpine); поддержка SSL/TLS (Let's Encrypt / certbot).
- Формат межсервисного взаимодействия:** JSON (REST через HTTP/1.1), Protocol Buffers v3 (gRPC/HTTP2).
- Внешние API:**
 - Telegram Bot API версии 6.x и выше;
 - ЮKassa REST API v3 (api.yookassa.ru).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

10. Поддерживаемые браузеры конечных пользователей (клиентский веб-интерфейс):

Таблица 4.5 — Поддерживаемые браузеры

Браузер	Минимальная версия
Google Chrome	90 и выше
Mozilla Firefox	88 и выше
Apple Safari	14 и выше
Microsoft Edge	90 и выше

4.6. Требования к маркировке и упаковке

Программный продукт поставляется исключительно в виде программного обеспечения; физические носители не предусмотрены.

- Маркировка образов контейнеров:** Docker-образы публикуются в GitLab Container Registry репозитория проекта и маркируются тегом вида `{сервис}:{semver}` (например, `events-api:1.0.0`). Каждый образ содержит метки (labels): `org.opencontainers.image.version`, `org.opencontainers.image.revision` (git-хэш), `org.opencontainers.image.created`.
- Поставка документации:** программная документация поставляется в двух форматах:
 - PDF — для ознакомления и передачи заказчику;
 - исходный код (Typst, расширение `.typ`) — для редактирования; хранится в каталоге `official_documents/` репозитория.
- Физическая упаковка:** поставка производится исключительно по сети (GitLab Container Registry); тара, упаковочные материалы и этикетки не применяются.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4.7. Требования к транспортированию и хранению

- Доставка образов:** Docker-образы доставляются с GitLab Container Registry командой `docker compose pull` по протоколу HTTPS. Аутентификация выполняется через переменные GitLab CI (`CI_REGISTRY_USER`, `CI_REGISTRY_PASSWORD`).
- Хранение исходного кода:** исходный код хранится в GitLab-репозитории под управлением Git. Ветка `main` является защищённой; прямые коммиты в неё запрещены; слияние выполняется через Merge Request с обязательным прохождением CI-пайплайна.
- Хранение данных:** постоянные данные хранятся в именованных томах Docker:
 - `postgres-data` — данные PostgreSQL (все базы данных платформы);
 - `redis-data` — данные Redis;
 - `minio-data` — объекты MinIO (QR-коды и медиафайлы).
- Резервное копирование:** резервная копия всех баз данных создаётся ежедневно средствами `pg_dump` и сохраняется в каталог `deploy/logs/backup/`. Срок хранения резервных копий — не менее 30 суток.
- Условия хранения серверного оборудования:** температура от +10°C до +35°C, влажность от 40 % до 80 % без конденсата; прямые солнечные лучи не допускаются.

4.8. Специальные требования

4.8.1. Требования к безопасности

- Шифрование транспортного уровня:** все внешние эндпоинты доступны исключительно по HTTPS (TLS 1.2 и выше); HTTP-запросы автоматически перенаправляются на HTTPS средствами Nginx.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. **Хэширование паролей:** пароли пользователей хранятся в базе данных только в виде хэша Argon2id (16 МБ памяти, 1 итерация, параллелизм 2); открытый текст не сохраняется и не передаётся ни в каком виде.
3. **Аутентификация без состояния на сервере (stateless JWT):** access-токены JWT не хранятся на сервере; их отзыв производится ожиданием истечения TTL (15 минут). Refresh-токены хранятся в базе данных и могут быть явно инвалидированы при выходе или смене пароля.
4. **Rate limiting:** Nginx ограничивает частоту запросов с одного IP-адреса по трём зонам; подробная конфигурация — таблица 4.9а.

Таблица 4.9а — Конфигурация ограничения частоты запросов (rate limiting)

Маршрут (location)	Зона	Лимит	Burst / nodelay
/api/v1/events	events_limit	300 з/с	300 / nodelay
/api/v1/auth	auth_limit	50 з/с	20 / nodelay
/api/v1/payment/	api_limit	100 з/с	20 / nodelay
/api/v1/registrations	api_limit	100 з/с	20 / nodelay
/api/v1/tickets	api_limit	100 з/с	20 / nodelay
/api/v1/promocodes	api_limit	100 з/с	10 / nodelay
/api/v1/checkin	api_limit	100 з/с	10 / nodelay

Зоны (limit_req_zone, ключ \$binary_remote_addr): auth_limit (10 МБ, 50 з/с), api_limit (10 МБ, 100 з/с), events_limit (10 МБ, 300 з/с). Превышение лимита возвращает ответ 429 Too Many Requests.

1. **Защита внутренних маршрутов:** доступ к интерфейсам Grafana, Swagger UI, Kafka UI и Redis Commander защищён HTTP Basic Auth (директива auth_basic в конфигурации Nginx).
2. **HTTP-заголовки безопасности** (применяются к HTTPS-серверу Nginx):
 - Strict-Transport-Security: max-age=63072000; includeSubDomains; preload — принудительное использование HTTPS на 2 года;
 - X-Frame-Options: SAMEORIGIN — защита от clickjacking;
 - X-Content-Type-Options: nosniff — запрет MIME-сниффинга;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

— X-XSS-Protection: 1; mode=block — дополнительная фильтрация XSS в браузере.

3. **Шифрование персональных данных в покое:** поля `telegram_username`, `first_name`, `last_name` в таблице `users` БД `auth_db` хранятся в зашифрованном виде (AES-шифрование на уровне приложения). Ключ шифрования задаётся через переменную окружения и не хранится в репозитории.
4. **Защита от OWASP Top 10:** при разработке применяются меры против следующих классов уязвимостей: SQL-инъекции (параметризованные запросы `pgx`), нарушенная аутентификация (JWT + 2FA), небезопасные прямые ссылки на объекты (IDOR) — проверка владельца ресурса, открытые редиректы — валидация `return_url`, XSS — экранирование вывода React.

4.8.2. Требования к производительности

1. **Время отклика REST API:** процентиль p95 времени ответа должен составлять не более 200 мс для эндпоинтов чтения и не более 800 мс для эндпоинта регистрации при нагрузке не менее 70 запросов/с.
2. **Пропускная способность Kafka:** система должна обрабатывать суммарно не менее 10 000 сообщений в минуту по всем топикам без деградации потребителей (Kafka consumer lag не превышает 1 000 сообщений).
3. **Количество одновременных соединений:** сервер должен выдерживать не менее 1 000 одновременных HTTP-соединений без возврата ошибок класса 5xx.

4.8.3. Требования к масштабируемости

1. Каждый компонент системы (API-сервер, воркер, сервис уведомлений) реализован в виде независимого контейнера без локального состояния (stateless), что позволяет горизонтально масштабировать каждый компонент независимо.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. PostgreSQL и Redis являются единственным источником разделяемого состояния; масштабирование операций чтения возможно через добавление реплик PostgreSQL.
3. Apache Kafka обеспечивает развязку производителей и потребителей; увеличение числа партиций топика позволяет линейно масштабировать число параллельных воркеров.
4. Использование объектного хранилища MinIO (S3-совместимый API) снимает ограничение на объём хранимых медиафайлов (QR-кодов) за счёт горизонтального расширения кластера.

4.8.4. Предполагаемые параметры нагрузки

Система должна обеспечивать корректную работу при следующих параметрах нагрузки:

Таблица 4.9 — Предполагаемые параметры нагрузки

Параметр	Значение
Одновременных онлайн-пользователей	до 1 000
Регистраций в сутки	до 10 000
Суммарных HTTP-запросов в сутки	до 500 000
Сообщений Kafka в минуту (пик)	до 10 000
Уведомлений через Telegram в минуту	до 200
Объём базы данных (12 месяцев)	до 10 ГБ
QR-кодов в объектном хранилище	до 100 000

4.8.5. Требования к CI/CD

Система непрерывной интеграции и доставки (CI/CD) реализуется на базе GitLab CI/CD и должна обеспечивать автоматизированный цикл сборки, тестирования и развёртывания всех компонентов платформы.

4.8.5.1. Стадии пайплайна

Пайплайн CI/CD состоит из следующих последовательных стадий:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1. Валидация (validate):

- проверка синтаксиса `docker-compose.yml` с подстановкой переменных окружения;
- компиляция всех Go-сервисов (`auth-manager`, `events-service`, `notifier`, `payment-service`, `qr-service`) с предварительной генерацией кода (`sqlc`, `oapi-codegen`, `protoc`).

2. Сборка образов (services-build): параллельная сборка Docker-образов всех сервисов (9 образов) инструментом Kaniko с публикацией в GitLab Container Registry. Каждый образ получает три тега: хэш коммита (`CI_COMMIT_SHORT_SHA`), версия (Git-тег при наличии) и `latest`.**3. Развёртывание Nginx (nginx-deploy):** запуск Nginx в HTTP-режиме для прохождения ACME-challenge (Let's Encrypt).**4. Настройка TLS (tls-setup):** получение SSL-сертификатов через certbot в режиме webroot; сохранение в именованные тома Docker.**5. Полное развёртывание (deploy:full):** поэтапный запуск инфраструктуры и сервисов с проверками здоровья после каждого этапа:

- инфраструктура (PostgreSQL, Redis, ZooKeeper, Kafka, MinIO, Prometheus, Grafana);
- создание баз данных (`auth_db`, `events_db`, `notifier_db`, `payment_db`);
- запуск всех сервисов приложения;
- ожидание статуса `healthy` всех контейнеров (таймаут — 180 с);
- перезагрузка Nginx с конфигурацией HTTPS.

6. Верификация (verify): автоматические smoke-тесты продакшн-эндпоинтов:

- GET /health (nginx);
- GET /api/v1/events;
- GET /api/v1/auth/health;
- GET /api/v1/payment/health;
- GET /monitoring/grafana/api/health.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4.8.5.2. Реестр контейнеров

Docker-образы публикуются в GitLab Container Registry проекта по адресу `registry.gitlab.com/cstati/cstati`. Схема именования образов: `{registry}/{service}:{version}`. Аутентификация при загрузке образов выполняется через переменные GitLab CI (`CI_REGISTRY_USER`, `CI_REGISTRY_PASSWORD`).

4.8.5.3. Self-hosted раннеры

Развёртывание системы выполняется через SSH-подключение к продакшн-серверу. Требования к раннерам и серверу:

Таблица 4.8 — Требования к раннерам и серверу развёртывания

Параметр	Значение
Способ развёртывания	SSH-подключение с GitLab CI-раннера к целевому серверу
Аутентификация	SSH-ключ (переменная <code>SSH_PRIVATE_KEY</code>)
Переменные окружения	<code>DEPLOY_USER</code> , <code>DEPLOY_SERVER</code> , <code>SSH_PRIVATE_KEY</code>
Инструмент сборки	Kaniko (сборка Docker-образов без Docker-in-Docker)
Целевая ОС сервера	Linux (Ubuntu 20.04 LTS и выше)
Docker Engine на сервере	версия 24.0 и выше
Docker Compose на сервере	версия 2.0 (Compose V2) и выше
Режим запуска заданий	Ручной (<code>when: manual</code>) для стадий развёртывания
Ветка развёртывания	<code>master</code> (защищённая)

4.8.5.4. Особенности процесса развёртывания

1. Все задания стадий развёртывания запускаются вручную (`when: manual`) для контроля момента выпуска.
2. Поддерживается чистое развёртывание (переменная `CLEAN_DEPLOY=true`): удаление именованных томов для полной переинициализации.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. При наличии действующих TLS-сертификатов их перевыпуск пропускается автоматически.
4. Файлы конфигурации (`deploy/`) передаются на сервер через `tar + ssh`, исключая дублирование в образах.
5. Предусмотрены задания обслуживания: `rebuild:infrastructure` (перезапуск инфраструктуры с опциональной очисткой данных) и `rebuild:services` (перезапуск сервисов с обновлённым тегом образа).

4.8.6. Требования к журналированию и диагностике

1. **Формат журналов:** все Go-сервисы (`auth-manager`, `events-service`, `notifier`, `payment-service`, `qr-service`) формируют структурированные журналы в формате JSON с полями: `time`, `level`, `msg`, `service`, `trace_id`; уровни: `DEBUG`, `INFO`, `WARN`, `ERROR`.
2. **Конфиденциальность:** открытый текст паролей, токенов и персональных данных (`telegram_username`, ФИО) не записывается в журналы ни при каких условиях.
3. **Доступ к журналам в production:** логи всех сервисов собираются Docker через `stdout/stderr` и доступны командой `docker compose logs {service}`. Дополнительно JSON-файлы записываются в каталог `deploy/logs/{service}/`.
4. **Профилирование:** каждый Go-сервис предоставляет эндпоинты `pprof` по пути `/debug/pprof/` для анализа потребления CPU и памяти; в `production` доступ закрыт Nginx (Basic Auth).
5. **Трассировка работоспособности:** все сервисы предоставляют `GET /health` (или `GET /api/{service}/health`) — эндпоинт, возвращающий HTTP 200 при штатной работе; используется Docker `healthcheck` и CI-верификацией.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4.8.7. Требования к отказоустойчивости

1. **Автовыключатель (Circuit Breaker):** `events-service` использует автовыключатель для вызовов к `auth-manager` (`gRPC Auth.Authenticate`). Параметры: порог ошибок 20 %, минимальное число запросов 20, интервал сброса 10 с. При открытом автовыключателе запросы, требующие аутентификации, возвращают `503 Service Unavailable`.
2. **Обработка ошибок в очереди (DLQ):** если воркер `events-service` не смог обработать сообщение из топиков `registrations` или `payments` после 5 попыток (экспоненциальная задержка 1 с, $\times 2$, макс. 30 с), сообщение публикуется в топик `registrations-dlq` для ручного разбора.
3. **Идемпотентность API:** эндпоинт `POST /api/v1/registrations` требует заголовок `Idempotency-Key: <UUID>`; повторный запрос с тем же ключом (в течение 24 ч) возвращает первоначальный результат из кэша Redis без создания дублирующей записи.
4. **Корректное завершение (Graceful Shutdown):** каждый Go-сервис перехватывает сигналы `SIGTERM/SIGINT` и дожидается завершения активных запросов (таймаут 30 с) перед остановкой.
5. **Проверки готовности:** Docker-образы всех сервисов содержат `HEALTHCHECK`; стадия CI `deploy:full` ожидает статус `healthy` от всех контейнеров (таймаут 180 с) перед подачей трафика.
6. **Надёжность Kafka-потребителей:** все потребители (`events-service-worker`, `payment-service`, `notifier-service`, `qr-service-group`) настроены на ручное подтверждение офсетов (`auto.commit.enable = false`); офсет подтверждается только после успешной обработки сообщения.

4.8.8. Формализованные endpoint-контракты (обязательные требования)

Ниже приведены обязательные контрактные требования к API-эндпоинтам. Контракт считается нарушенным, если отклоняется хотя бы один из пунктов: структура JSON, обязательность полей, коды ответа, SLA по времени ответа, требования авторизации.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4.8.8.1. Контракты auth-manager

Таблица 4.10 — Контрактные требования к endpoint auth-manager

Endpoint	Вход (минимум)	Успех	Ошибки	SLA
POST /api/v1/auth/sign_up	username, password, telegram_username	201 Created, {id, username}	400, 409, 429	p95 <= 300 мс
POST /api/v1/auth/sign_in	username, password, user_agent	200, {session_id, status}	401, 423, 429	p95 <= 250 мс
POST /api/v1/auth/sessions/{id}/submit_2fa_code	code (6 цифр)	200, {access_token, refresh_token}	400, 401, 410, 429	p95 <= 250 мс
POST /api/v1/auth/refresh_token	refresh_token	200, {access_token, refresh_token}	401, 403, 429	p95 <= 250 мс

4.8.8.2. Контракты events-service

Таблица 4.11 — Контрактные требования к endpoint events-service

Endpoint	Вход (минимум)	Успех	Ошибки	SLA
GET /api/v1/events	skip, limit, status?	200, {items[], total}	400, 429	p95 <= 150 мс
POST /api/v1/events	name, slug, start_date, end_date, allowed_payment_types	201, {event_id}	400, 401, 403, 409, 429	p95 <= 350 мс
POST /api/v1/registrations	Idempotency-Key, fio, email, tg, ticket_id, payment_method	202, {registration_id, status}	400, 401, 404, 409, 429	p95 <= 800 мс

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Endpoint	Вход (минимум)	Успех	Ошибки	SLA
POST /api/v1/promocodes/{code}/activate	wave_id	200, {promocode_id, discount_percent, expires_at}	400, 404, 409, 429	p95 <= 200 мс
POST /api/v1/checkin/{registration_id}	path registration_id	200, {status: checked}	401, 403, 404, 409, 429	p95 <= 250 мс

4.8.8.3. Контракты payment-service и qr-service

Таблица 4.12 — Контрактные требования к endpoint payment-service и qr-service

Endpoint	Вход	Успех	Ошибки	SLA
POST /api/v1/payment/webhook	Webhook JSON ЮKassa	200	400, 401, 403, 409, 500	p95 <= 500 мс
GET /api/v1/payment/success	registration_id	200, HTML/JSON статуса	400, 404, 409	p95 <= 200 мс
GET /api/qr/health	без тела	200, {status: ok}	503	p95 <= 50 мс
GET /api/notifier/health	без тела	200, {status: ok}	503	p95 <= 50 мс

4.8.9. Матрица ошибок и единый формат отказов

Для всех HTTP API обязателен единый формат ошибки:

```
{
  "error": {
    "code": "STRING_CODE",
    "message": "человекочитаемое описание",
    "details": {...},
    "trace_id": "..."
  }
}
```

Таблица 4.13 — Матрица ошибок API и правила обработки

HTTP код	Причина	Поведение сервера	Действие клиента
400 Bad Request	Нарушен контракт входных данных	Валидация отклоняет запрос, запись не создается	Исправить данные и повторить

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

HTTP код	Причина	Поведение сервера	Действие клиента
401 Unauthorized	Отсутствует или неверен JWT / подпись webhook	Доступ запрещен, состояние не меняется	Повторить с валидной авторизацией
403 Forbidden	Недостаточно прав (не владелец/не админ)	Отказ без изменения состояния	Запросить доступ
404 Not Found	Сущность отсутствует	Отказ без создания сущностей	Проверить идентификатор
409 Conflict	Конфликт состояния: повторный check-in, дубликат slug, гонка версий	Возврат текущего состояния	Перечитать сущность и повторить
410 Gone	2FA/lock истек по TTL	Сессия/лок помечается истекшим	Начать сценарий заново
429 Too Many Requests	Сработал rate-limit	Запрос отклоняется, логируется лимитер	Повторить с backoff
500 Internal Server Error	Внутренняя ошибка обработки	Rollback транзакции, событие в лог и метрики	Повторить позже
503 Service Unavailable	Недоступен dependency / открыт circuit breaker	Fail-fast, без записи	Ретрай по экспоненте

4.8.10. State-machine переходы доменных сущностей

4.8.10.1. Автомат состояний регистрации

Допустимые состояния registration:

pending -> confirmed -> paid -> checked_in

Допустимые аварийные/альтернативные переходы:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

pending -> expired, pending -> failed, pending -> cancelled, confirmed -> cancelled, paid -> cancelled (только через административную процедуру возврата).

Запрещенные переходы (должны давать 409):

- checked_in -> paid/confirmed/pending;
- expired -> paid;
- failed -> paid без новой регистрации;
- повторный checked_in -> checked_in.

Таблица 4.14 — State-machine регистрации: события и переходы

Текущее состояние	Событие	Новое состояние	Исполнитель
pending	payment_succeeded	paid	PaymentWorker
pending	payment_failed	failed	PaymentWorker
pending	ttl_expired	expired	ExpirationWorker
paid	checkin_confirmed	checked_in	events-api
confirmed	manual_cancel	cancelled	admin

4.8.10.2. Автомат состояний волны регистрации

Допустимые состояния wave: scheduled, active, paused, sold_out, closed, expired.

Разрешенные переходы:

- scheduled -> active (по времени или ручной командой);
- active -> paused;
- paused -> active;
- active -> sold_out (достигнут лимит);
- active|paused|sold_out -> closed (ручное закрытие);
- scheduled|active|paused -> expired (истечение окна регистрации).

Все остальные переходы считаются недопустимыми и должны отвергаться с 409.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4.8.10.3. Автомат статуса платежа

Состояния payment: new, waiting_for_capture, succeeded, canceled, failed.

Ключевые правила:

- событие succeeded является терминальным;
- повторное поступление succeeded должно быть идемпотентным и не изменять данные повторно;
- canceled/failed терминальны и не могут переходить в succeeded без создания новой транзакции.

4.8.11. Сценарии отказов и восстановление

Таблица 4.15 — Сценарии отказов и требования к восстановлению

Сценарий	Ожидаемое поведение	Потери данных	RTO/RPO
Недоступен Kafka broker	HTTP API продолжает принимать чтение; запись регистраций переводится в controlled failure 503/500; алерт firing	0, если транзакция не подтверждена	RTO ≤ 15 мин, RPO = 0
Недоступен PostgreSQL	Все state-changing операции fail-fast; rollback; health=503	0	RTO ≤ 30 мин, RPO ≤ 5 мин при аварии диска
Недоступен Redis	Идемпотентность работает в degraded-режиме с DB-check fallback; latency увеличивается	0	RTO ≤ 15 мин, RPO = 0
Недоступен auth-manager	Защищенные endpoint возвращают 503 через circuit breaker	0	RTO ≤ 15 мин, RPO = 0
Недоступен ЮKassa	payment-service ставит retry и не меня-	0	RTO зависит от внешнего сервиса

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Сценарий	Ожидаемое поведение	Потери данных	RTO/RPO
	ет терминальные статусы без подтверждения		
Недоступен MinIO	qr-service не отправляет уведомление до успешной записи, повторяет задачу	0	RTO <= 30 мин, RPO = 0

Обязательные требования к rollback/recovery:

- все транзакционные операции должны быть повторяемыми без дублирования результата;
- recovery-процедура должна включать повторную обработку DLQ и сверку консистентности registration/payment/ticket;
- после восстановления должна выполняться автоматическая smoke-проверка ключевого сценария registration -> payment -> qr -> check-in.

4.8.12. Требования к мониторингу, SLI/SLO и алертингу

Для каждого критического компонента устанавливаются обязательные индикаторы SLI, целевые SLO и пороги алертов.

Таблица 4.16 — SLI/SLO и правила алертинга

Компонент	SLI	SLO	Alert rule
events-api	p95 latency	<= 300 мс для read, <= 800 мс для registration	warning > SLO 10 мин; critical > SLO 30 мин
events-worker	consumer lag	< 1000 сообщений	critical при lag > 5000 более 5 мин
payment-service	webhook success ratio	>= 99.5%	critical при error ratio > 2% за 10 мин
auth-manager	auth success ratio	>= 99.9%	critical при 5xx > 1% за 5 мин
qr-service	qr generation duration p95	<= 800 мс	warning при p95 > 1200 мс 10 мин

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Компонент	SLI	SLO	Alert rule
notifier	delivery success ratio	$\geq 99.0\%$	critical при fail ratio $> 5\%$ за 15 мин
postgres	connection saturation	$< 80\%$ pool usage	critical $> 95\%$ 5 мин
redis	hit ratio	$\geq 80\%$	warning $< 60\%$ 15 мин
nginx	5xx ratio	$< 0.5\%$	critical $> 1\%$ 5 мин

Обязательные метрики (minimum set):

- HTTP: http_requests_total, http_request_duration_seconds, http_5xx_total;
- Kafka: kafka_consumer_lag, kafka_messages_processed_total, kafka_dlq_total;
- Биз-
нес: registrations_created_total, payments_succeeded_total, payments_failed_total, checkin_total;
- Инфраструктура: db_pool_in_use, redis_connected_clients, container_cpu_usage_seconds_total, container_memory_usage_bytes.

Обязательные алерты (minimum set):

- HighErrorRate;
- HighLatencyP95;
- KafkaLagCritical;
- DLQGrowthDetected;
- PaymentWebhookFailureBurst;
- AuthServiceUnavailable;
- DatabasePoolExhaustion;
- Nginx5xxCritical.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

5. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

5.1. Состав программной документации

Разработка комплекса программ CSTATI.events должна сопровождаться следующей программной документацией в соответствии с ГОСТ 19.201–78:

Документы, разрабатываемые командой:

1. **Техническое задание** (ГОСТ 19.201–78) — настоящий документ, определяющий требования к функциональным характеристикам, эксплуатационным параметрам, технико-хозяйственным требованиям, порядку контроля и приёмки.
2. **Программа и методика испытаний** (ГОСТ 19.301–79) — документ, описывающий методы проверки соответствия разработанного ПО требованиям ТЗ; включает план тестирования, сценарии тестов, требования к стендам.

Документы, разрабатываемые каждым участником:

3. **Текст программы** (ГОСТ 19.401–78) — исходный код микросервисов, форматированный и задокументированный в соответствии с договорённостями команды.
4. **Пояснительная записка** (ГОСТ 19.404–79) — описание архитектуры, дизайн-решений, обоснование выбора технологий, анализ сложности реализованных алгоритмов, описание тестирования.
5. **Руководство оператора** (ГОСТ 19.505–79) — инструкции по развёртыванию, настройке параметров окружения, запуску сервисов, мониторингу и обслуживанию системы в продакшене.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

5.2. Специальные требования к программной документации

5.2.1. Требования к формату и оформлению

1. Все документы должны быть выполнены в соответствии с ГОСТ 19.106–78 (межгосударственная система документирования ПО);
2. Пояснительные записки оформляются на листах А4 с полями: левое поле не менее 30 мм, правое не менее 10 мм, верхнее и нижнее не менее 20 мм;
3. Текст размещается через полуторный интервал (line-height 1.5), шрифт Times New Roman 12 pt или эквивалент;
4. Нумерация разделов, подразделов, таблиц, рисунков выполняется в соответствии с ГОСТ 19.106–78;
5. Все документы должны содержать: титульный лист, оглавление, выходные данные (код по ГОСТ 19.103–77, дату, подписи);
6. Архив с электронными копиями документов упаковывается в формате .zip или .rar, содержит всю документацию в формате .pdf и исходный код программ.

5.2.2. Требования к содержанию документации

Пояснительная записка должна включать:

- описание предметной области и постановку задачи;
- анализ существующих решений и обоснование разработки нового ПО;
- архитектурное описание системы: микросервисная архитектура, взаимодействие компонентов, используемые паттерны проектирования;
- описание используемых технологий: языки программирования, фреймворки, базы данных, очереди сообщений, инструменты сборки;
- описание алгоритмов обработки ключевых операций (например, атомарное резервирование мест, идемпотентность платежей);
- анализ сложности и производительности критических операций;
- описание системы тестирования и покрытие кода (unit-тесты, интеграционные тесты, нагрузочные тесты);

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- инструкции по развёртыванию и запуску на локальной машине и в продакшене;
- результаты тестирования: отчёты, логи, метрики покрытия;
- заключение о соответствии требованиям ТЗ.

Руководство оператора должно включать:

- системные требования (ОС, версии ПО, объём памяти и диска);
- инструкции по развёртыванию в Docker;
- инструкции по настройке окружения (переменные окружения, конфигурационные файлы);
- инструкции по запуску, остановке, перезагрузке сервисов;
- инструкции по просмотру логов и диагностике ошибок;
- инструкции по резервному копированию баз данных;
- инструкции по масштабированию (добавление реплик контейнеров);
- инструкции по обновлению версии ПО (миграция данных, откат);
- контакты для технической поддержки;
- приложение с примерами конфигураций для разных окружений.

Текст программы (исходный код) должен:

- содержать комментарии на русском или английском языке, объясняющие сложные части кода;
- использовать осмысленные имена переменных, функций и типов;
- содержать примеры вызовов для публичных API;
- содержать информацию о лицензии, авторе, дате последнего изменения в виде заголовка файла;
- соответствовать стандартам форматирования языка (для Go: `gofmt`, для Python: `black`, для JavaScript: `prettier`);
- содержать обработку ошибок и логирование ключевых операций.

5.2.3. Требования к электронной доставке

1. Пояснительная записка загружается в систему проверки на плагиат (Антиплагиат ВШЭ) не позже, чем за день до защиты;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. Лист из системы проверки подтверждающий загрузку сдаётся в учебный офис вместе со всеми материалами;
3. Все документы (ТЗ, ПМИ, ТП, ПЗ, РО, РИ) сдаются в электронном виде в архиве .zip/.rar с чёткой структурой папок;
4. Исходный код программ и утилиты сборки включаются в тот же архив в папке /source_code/.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

6. ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

6.1. Экономическая эффективность

Разработка Платформы выполняется в рамках учебного курсового проекта по дисциплине «Программная инженерия»; коммерческая оценка стоимости разработки и сопровождения не является целью настоящего технического задания. Тем не менее применение Системы обеспечивает ряд измеримых технико-экономических преимуществ по сравнению с организацией мероприятий вручную.

6.1.1. Снижение трудозатрат организаторов

При отсутствии специализированного программного средства проведение публичного технического мероприятия сопряжено со следующими ручными трудозатратными операциями, каждая из которых требует отдельного исполнителя и значительного рабочего времени:

- 1) ручной сбор заявок на участие через формы в мессенджерах, по адресу электронной почты или через сторонние сервисы без интеграции с системой оплаты;
- 2) ручная сверка банковских поступлений со списком зарегистрированных участников и ручное обновление статуса каждого участника;
- 3) ручная рассылка подтверждений регистрации, напоминаний о мероприятии и ссылок на оплату через мессенджеры или электронную почту;
- 4) ручная верификация участников при входе на мероприятие по распечатанным спискам или телефонным базам данных;
- 5) ручной учёт использованных промокодов и скидочных предложений.

Применение Платформы CSTATI позволяет полностью автоматизировать каждую из перечисленных операций. По оценке, основанной на сравнении с аналогичными решениями на рынке event-технологий, суммарные трудозатраты организатора на подготовку и проведение мероприятия сокращаются ориентировочно на 60–80 % при типовой аудитории от 100 до 500 участников.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

7. СТАДИИ И ЭТАПЫ РАЗРАБОТКИ

Стадии и этапы разработки определены в соответствии с ГОСТ 19.102–77.

Таблица 7.1 — Стадии и этапы разработки

Стадии разработки	Этапы работ	Содержание работ	Сроки
Техническое проектирование	Анализ требований	Составление ТЗ, согласование с руководителем	До 15.01.26
	Проектирование архитектуры	Выбор технологий, проектирование баз данных, API, интерфейсов	До 15.01.26
Рабочее проектирование	Разработка backend	Реализация сервисов авторизации, мероприятий, оплаты, Telegram-бота	До 15.03.26
	Разработка frontend	Вёрстка, компоненты, личный кабинет, административная панель	До 15.03.26
	Интеграция и тестирование	Сборка системы, модульное и интеграционное тестирование	До 15.03.26
Внедрение и испытания	Развёртывание	Настройка серверов, доменов, SSL, деплой	До 01.04.26
	Приёмочные испытания	Тестирование на соответствие ТЗ, нагрузочное тестирование	До 01.04.26
	Подготовка документации	Пояснительная записка, руководства, итоговый отчёт	До 01.04.26
Сдача проекта	Защита проекта	Презентация, демонстрация, ответы на вопросы	

7.1. Сроки разработки и исполнители

Программный продукт (программа и документация) должен быть завершён не позднее 30.04.2026 — даты, утверждённой путём согласования заказчиком и руководителем проекта.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

8. ПОРЯДОК КОНТРОЛЯ И ПРИЁМКИ

8.1. Критерии приёмки

Система CSTATI.events считается готовой к приёмке, если выполнены все следующие критерии:

1. Критерии качества кода:

- все unit-тесты проходят без ошибок: `go test ./...` (или аналог для Python/JavaScript);
- покрытие кода тестами $\geq 80\%$ для бизнес-логики: `go tool cover -func=coverage.out`;
- результаты анализа кода статическими анализаторами приемлемы: `golangci-lint run ./...` без критических ошибок;
- форматирование кода соответствует стандартам: `gofmt -l .` (for Go).

2. Критерии функциональности:

- все REST API-эндпоинты, перечисленные в разделе 4 ТЗ, реализованы и доступны;
- все gRPC-методы сервисов аутентификации и уведомлений реализованы;
- регистрация пользователей работает с двухфакторной аутентификацией;
- регистрация участников на мероприятие работает с применением промокодов;
- интеграция с платёжным шлюзом ЮKassa работает корректно (платёжные ссылки создаются, webhook'и принимаются);
- генерация QR-кодов работает и загруженные изображения доступны в MinIO;
- уведомления через Telegram доставляются участникам.

3. Критерии интеграции:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- микросервисы взаимодействуют друг с другом через определённые протоколы (HTTP REST, gRPC, Kafka);
- полный сценарий «регистрация → подтверждение → оплата → Telegram-билет → check-in» работает без сбоев;
- обработчики Kafka (RegistrationWorker, PaymentWorker, ExpirationWorker) в events-service корректно обрабатывают сообщения.

4. Критерии производительности:

- время отклика REST API на базовые операции ≤ 500 мс (p95 / p99) при нагрузке до 100 одновременных пользователей;
- Kafka пропускает $\geq 10\,000$ сообщений/мин без потерь;
- система успешно обслуживает 50+ мероприятий одновременно.

5. Критерии надёжности:

- все операции с состоянием (платежи, резервирование мест) идиempotentны и отказоустойчивы;
- при отключении зависимого сервиса (Kafka, Redis) не происходит потеря данных о регистрации или оплате;
- сообщения, не обработанные корректно, направляются в DLQ-топик registrations-dlq;
- система восстанавливается после перезагрузки контейнеров без потери состояния.

8.2. Процедура приёмки

1. **Функциональное тестирование** — проверка всех функциональных требований раздела 4 путём выполнения тест-сценариев из Программы и методики испытаний.
2. **Интеграционное тестирование** — проверка корректного взаимодействия микросервисов, наличия необходимых Kafka-топиков, маршрутизации HTTP-запросов через Nginx.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. **Модульное тестирование кода** — запуск набора unit-тестов, проверка покрытия кода $\geq 80\%$.
4. **Нагрузочное тестирование** — проверка производительности при повышенной нагрузке (500+ одновременных регистраций за минуту) с использованием инструмента k6 или локального скрипта.
5. **Тестирование надёжности** — проверка обработки сбоев (отключение БД, Kafka, Redis), отсутствие потери данных, корректная работа механизмов повторных попыток.
6. **Проверка безопасности** — валидация JWT-токенов, проверка разграничения доступа по ролям, тестирование на уязвимости OWASP Top 10 (SQL-инъекции, XSS, открытые редиректы).
7. **Составление акта приёмки** — по результатам всех тестов составляется акт приёмки подписываемый сторонами.

8.3. Специальные требования к тестированию для events-service

События-сервис (events-service) является ключевым компонентом системы и должен пройти углубленное тестирование:

1. Unit-тесты:

- проверка логики создания, редактирования и удаления мероприятий;
- проверка логики управления волнами регистрации и их статусов;
- проверка логики применения промокодов и расчёта скидок;
- проверка логики атомарного резервирования мест с оптимистичной блокировкой;
- проверка обработки граничных случаев (исчерпаны все места, истек срок регистрации, некорректный промокод).

2. Интеграционные тесты:

- проверка взаимодействия с PostgreSQL: создание регистраций, чтение, обновление статусов, откат при ошибках;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- проверка взаимодействия с Kafka: публикация событий в топике `registrations`, `payments`, `qr-generation`; проверка `RegistrationWorker`, `PaymentWorker`, `ExpirationWorker`;
- проверка взаимодействия с Redis: сохранение и извлечение `Idempotency-Key` для кэширования;
- проверка полного цикла регистрации: запрос на регистрацию → сохранение в БД → публикация в Kafka → обработчик → обновление статуса → Telegram-уведомление.

3. Тесты API-эндпоинтов:

- проверка аутентификации (только авторизованные пользователи могут создавать мероприятия);
- проверка пагинации списка регистраций (параметры `skip`, `limit`, `page_size`);
- проверка обработки некорректных входных данных (валидация `email`, `length strings`, типов);
- проверка лимитирования частоты запросов на активацию промокодов (max 5 запросов/мин с одного IP).

Все вышеперечисленные тесты уже реализованы в репозитории и должны проходить без ошибок.

8.4. Требования к результатам приёмки

1. Все юнит-тесты должны проходить: `go test ./... -cover` должна вывести статус PASS для всех пакетов.
2. Покрытие кода бизнес-логики не менее 80%, что подтверждается отчётом: `go tool cover -html=coverage.out`.
3. Интеграционные тесты с действующими Kafka, PostgreSQL, Redis должны проходить полностью.
4. Отсутствие критических отклонений, определяемых как:
 - потеря или повреждение данных регистрации или платежей;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- невозможность завершить полный цикл мероприятия (от создания до check-in);
- недоступность хотя бы одного критического сервиса (events-service, auth-manager, payment-service) более чем на 5 минут.

5. На основании успешного прохождения всех испытаний составляется и подписывается акт приёмки программного комплекса.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ГЛОССАРИЙ И СОКРАЩЕНИЯ

Термины и сокращения, используемые в настоящем документе:

API — Application Programming Interface — программный интерфейс взаимодействия.

CI/CD — Continuous Integration / Continuous Deployment — практика непрерывной интеграции и доставки программного обеспечения.

DLQ — Dead Letter Queue — очередь недоставленных сообщений в Kafka для сообщений, обработка которых завершилась неудачей после исчерпания попыток повтора.

gRPC — Google Remote Procedure Call — высокопроизводительный RPC-фреймворк на базе Protocol Buffers и HTTP/2.

JWT — JSON Web Token (RFC 7519) — компактный, самодостаточный токен для передачи утверждений между сторонами в формате JSON.

QR-код — Quick Response Code — двумерный штриховой код стандарта ISO/IEC 18004, используемый в Системе для идентификации и валидации билета участника.

REST — Representational State Transfer — архитектурный стиль построения распределённых гипермедиасистем на основе HTTP.

SPA — Single-Page Application — одностраничное веб-приложение, динамически обновляющее содержимое страницы без перезагрузки.

SSL/TLS — Secure Sockets Layer / Transport Layer Security — протоколы криптографической защиты сетевого соединения.

2FA — Двухфакторная аутентификация — механизм дополнительной проверки личности пользователя через одноразовый код, доставляемый по независимому каналу.

Kafka — Apache Kafka — распределённый журнал сообщений (commit log), используемый как брокер для асинхронного взаимодействия микросервисов.

Комплекс программ — Вид программного изделия по ГОСТ 19.101-77, §2.2: два и более взаимодействующих программных компонента, выполняющих совместную функцию.

Микросервис — Самостоятельный, независимо развёртываемый компонент системы, реализующий ограниченный функциональный контекст (bounded context).

MinIO — S3-совместимое объектное хранилище с открытым исходным кодом, используемое для хранения QR-кодов и медиафайлов.

Nginx — Высокопроизводительный веб-сервер и обратный прокси, выполняющий функции API Gateway в Системе.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

PostgreSQL — Реляционная СУБД с открытым исходным кодом (версия 16), используемая как основное хранилище данных всех сервисов.

Prometheus — Система мониторинга и сбора временных рядов (time-series) метрик с открытым исходным кодом.

Redis — Высокопроизводительное in-memory хранилище типа ключ-значение, используемое для кэширования идемпотентных ключей и снижения нагрузки на БД.

ЮKassa — Российский платёжный шлюз, предоставляемый ООО «ЮМани», используемый для приёма онлайн-платежей банковскими картами в Системе.

ЕСПД — Единая система программной документации — комплекс государственных стандартов ГОСТ 19.xxx, устанавливающих правила разработки, оформления и обращения программной документации.

CSTATI — Краткое наименование разработки: микросервисная платформа управления мероприятиями.

Check-in — Процедура подтверждения прохода участника на мероприятие путём валидации QR-кода его электронного билета.

Волна регистрации — Ограниченный по времени период открытой регистрации в рамках мероприятия; одно мероприятие может иметь несколько последовательных волн с разными ценами и лимитами мест.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 2

ПЕРЕЧЕНЬ КАФКА-ТОПИКОВ

Таблица 2.1 — Kafka-топики системы

Топик	Парт.	Производитель	Потребитель (Consumer Group)
registrations	3	events-api	events-worker (event-service-worker-prod)
payments	3	events-worker	payment-service
notifications	3	events-worker, payment-service, tg-bot	notifier (notifier-service)
ticket-status-updates	1	events-worker	events-api
registrations-dlq	1	events-worker (при ошибках)	ручная обработка / алерты
qr-generation	3	events-worker	qr-service

Примечание. Retention всех топиков — 168 часов (7 суток). Replication factor = 1 (одиночный брокер в dev/prod конфигурации).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 3

ПЕРЕЧЕНЬ ОСНОВНЫХ API-ЭНДПОИНТОВ

Events Service (порт 8000)

Таблица 3.1 — REST API Events Service

Метод	Путь	Авторизация	Описание
GET	/api/v1/events	—	Список мероприятий (пагинация)
POST	/api/v1/events	JWT+Admin	Создание мероприятия
GET	/api/v1/events/{slug}	—	Мероприятие по slug
PATCH	/api/v1/events/{id}	JWT+Admin	Изменение мероприятия
POST	/api/v1/events/{id}/waves	JWT+Admin	Создание волны
GET	/api/v1/events/{id}/waves	JWT+Admin	Список волн
PATCH	/api/v1/events/{id}/waves/{wid}	JWT+Admin	Изменение волны
POST	/api/v1/events/{id}/tickets	JWT+Admin	Создание билета
POST	/api/v1/events/{id}/promocodes	JWT+Admin	Создание промокода
POST	/api/v1/promocodes/{code}/activate	—	Активация промокода
POST	/api/v1/registrations	—	Регистрация участника
GET	/api/v1/events/{id}/guests	JWT+Admin	Список гостей
POST	/api/v1/registrations/{id}/checkin	JWT+Admin	Check-in участника
GET	/metrics	—	Метрики Prometheus

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Auth Manager (порт 8001)

Таблица 3.2 — REST API Auth Manager

Метод	Путь	Описание
POST	/api/v1/auth/sign_up	Регистрация пользователя
POST	/api/v1/auth/sign_in	Начало сессии входа (→ sessionID)
GET	/api/v1/auth/sessions/{sessionID}	Метаданные сессии
POST	/api/v1/auth/sessions/{sessionID}/submit_2fa_code	Подтверждение кода 2FA (→ токены)
POST	/api/v1/auth/sessions/{sessionID}/resend_2fa_code	Повторная отправка кода 2FA
POST	/api/v1/auth/refresh_token	Обновление access-токена
GET	/api/v1/auth/users/{userID}	Данные пользователя (JWT)
GET	/api/v1/auth/users	Список пользователей (JWT)
GET	/api/v1/auth/health	Healthcheck
GET	/metrics	Метрики Prometheus

Payment Service (порт 8004)

Таблица 3.3 — REST API Payment Service

Метод	Путь	Описание
POST	/api/v1/payment/webhook	Webhook от ЮKassa (статус платежа)
GET	/api/v1/payment/success	Редирект пользователя после оплаты
GET	/api/v1/payment/health	Healthcheck
GET	/metrics	Метрики Prometheus

Notifier (порт 8002)

Таблица 3.4 — REST API Notifier

Метод	Путь	Авторизация	Описание
GET	/api/notifier/health	—	Healthcheck
GET	/metrics	—	Метрики Prometheus

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Примечание. Основное взаимодействие с Notifier производится через gRPC-интерфейс `NotifierService.SendNotification()` (порт 50052) и через Kafka-топик `notifications (consumer group notifier-service)`. HTTP-эндпоинты сервиса ограничены `healthcheck` и метриками.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ПРИЛОЖЕНИЕ 4**СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ**

1. ГОСТ 19.101–77 Виды программ и программных документов // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102–77 Стадии разработки // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103–77 Обозначения программ и программных документов // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104–78 Основные надписи // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105–78 Общие требования к программным документам // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106–78 Требования к программным документам, выполненным печатным способом // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201–78 Техническое задание. Требования к содержанию и оформлению // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.603–78 Общие правила внесения изменений // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.604–78 Правила внесения изменений в программные документы, выполненные печатным способом // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.301–79 Программа и методика испытаний // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.401–78 Текст программы // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

12. ГОСТ 19.404–79 Пояснительная записка // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
13. ГОСТ 19.505–79 Руководство оператора // Единая система программной документации. — М.: ИПК Издательство стандартов, 2001.
14. The Go Programming Language Specification. Официальная документация языка программирования Go (версия 1.24). — URL: <https://go.dev/ref/spec> (дата обращения: 20.04.2026).
15. Python Language Reference. Официальная документация языка программирования Python (версия 3.10). — URL: <https://docs.python.org/3/> (дата обращения: 20.04.2026).
16. React Documentation. Официальная документация библиотеки React (версия 18.3). — URL: <https://react.dev/> (дата обращения: 20.04.2026).
17. PostgreSQL 16 Documentation. Официальная документация системы управления реляционными базами данных PostgreSQL. — URL: <https://www.postgresql.org/docs/16/> (дата обращения: 20.04.2026).
18. Redis Documentation. Официальная документация системы кэширования и хранилища данных Redis (версия 7). — URL: <https://redis.io/docs/> (дата обращения: 20.04.2026).
19. Apache Kafka Documentation. Официальная документация распределённого брокера сообщений Apache Kafka. — URL: <https://kafka.apache.org/documentation/> (дата обращения: 20.04.2026).
20. Docker Documentation. Официальная документация платформы контейнеризации Docker (версия 24). — URL: <https://docs.docker.com/> (дата обращения: 20.04.2026).
21. Nginx Documentation. Официальная документация веб-сервера и обратного прокси Nginx (версия 1.25). — URL: <https://nginx.org/en/docs/> (дата обращения: 20.04.2026).
22. Jones M., Bradley J., Sakimura N. RFC 7519: JSON Web Token (JWT). IETF, 2015. — URL: <https://tools.ietf.org/html/rfc7519> (дата обращения: 20.04.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

23. MinIO Documentation. Официальная документация объектного хранилища MinIO (S3-совместимое). — URL: <https://min.io/docs/> (дата обращения: 20.04.2026).
24. ЮKassa API Reference. Официальная документация платёжного шлюза ЮKassa (версия 3). — URL: <https://yookassa.ru/developers/api> (дата обращения: 20.04.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.09.01-01 ТЗ				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]